

Markov Decision Processes and Partial Observability

Lecture 23

CS 4880

Kirk Duran

Last Time: One-Step Decision Making

The robot combined three ingredients:

- ▶ a belief about the current world;
- ▶ a utility model over outcomes;
- ▶ expected utility to choose the best action now.

$$a^* = \arg \max_a EU(a)$$

Key idea

Decision networks solved “what should I do now?” but did not yet reason about how today’s action changes tomorrow’s choices.

The Problem: A Locally Good Action Can Create a Bad Future

Shortcut now

Immediate reward: +6.

Fast if the plaza stays clear, but entering it may trap the robot behind dense pedestrian traffic.

Detour now

Immediate reward: +2.

Slower right now, but it moves the robot into a predictable corridor with many good future choices.

Key idea

The quality of an action depends on both its immediate outcome and the future states it makes likely.

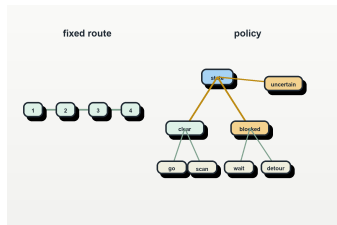
This Is No Longer Just a Path Problem

A path is a fixed sequence:

North, North, East, East, . . .

But the robot does not control exactly what happens after each move.

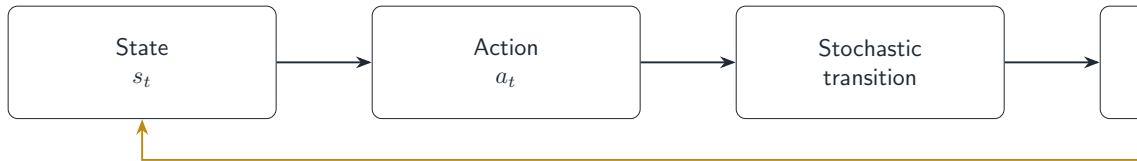
- ▶ pedestrians change motion;
- ▶ doors may open or close;
- ▶ wheel slip changes position;
- ▶ congestion changes what is reachable next.



Key idea

We need a policy—a decision rule that reacts to the state reached, not a single precomputed path.

The MDP Loop



Key idea

A Markov Decision Process formalizes repeated decision making when actions have uncertain consequences and future rewards matter.

A Markov Decision Process

$$MDP = (S, A, T, R, \gamma)$$

S	states: what situation is the robot in?
A	actions: what can the robot do?
T	transition model: where might an action lead?
R	reward: what immediate feedback is produced?
γ	discount factor: how strongly do future rewards matter?

Key idea

An MDP separates the world model from the decision objective, then connects them through a policy.

What Counts as a State?

A useful robot state might include:

- ▶ location or campus region;
- ▶ battery band;
- ▶ remaining delivery task;
- ▶ whether a door is open;
- ▶ congestion level if it changes future transitions.

The Markov property is

$$P(s_{t+1} \mid s_t, a_t, \text{history}) = P(s_{t+1} \mid s_t, a_t).$$

Key idea

“State” is an engineering abstraction: include what matters for predicting the future, omit what does not.

State Design Is Already a Tradeoff

State too small

If state is only location, but battery changes which moves are safe, then two visits to the “same” state can have different futures. The model is not truly Markov.

State too rich

Location $\times$ battery $\times$ task $\times$ crowd density $\times$ weather $\times$ door status $\times \dots$
Prediction improves, but the number of states explodes.

Key idea

Markov state design trades model fidelity against state-space size.

Actions Are the Robot's Controllable Choices

The action set might include:

- ▶ move forward, turn, backtrack;
- ▶ wait for traffic to move;
- ▶ perform a slower scan or re-localize;
- ▶ deliver the package;
- ▶ dock or request assistance.

Key idea

The action set defines what control authority the agent actually has.

The Transition Model Makes Actions Stochastic

$$T(s, a, s') = P(s_{t+1} = s' \mid s_t = s, a_t = a)$$

Example from the campus intersection:

Action	Possible next states
Shortcut	0.70 NearGoal, 0.25 CrowdedPlaza, 0.05 SameLocation
Detour	0.95 SafeCorridor, 0.05 SameLocation
Wait	0.60 CrowdClears, 0.40 CrowdPersists

Key idea

The robot chooses the action, not the outcome.

Reward Is Immediate; Return Is Long-Term

Reward R_t

Feedback for one transition.

- ▶ +20 deliver package
- ▶ -1 each time step
- ▶ -8 enter dense crowd
- ▶ -100 collision

Return G_t

Accumulated reward produced by a sequence of future decisions.

A move can be mediocre now but create a much better long-term future.

Key idea

MDPs optimize a trajectory of consequences, not the next reward in isolation.

Discounted Return

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

- ▶ $\gamma = 0$: only the next reward matters.
- ▶ $0 < \gamma < 1$: future rewards matter but are progressively discounted.
- ▶ $\gamma \approx 1$: long-run consequences matter strongly.

Key idea

The discount factor controls how strongly delayed consequences influence current actions.

What Does the Discount Factor Mean for the Robot?

Time preference

Earlier task completion may be operationally more valuable.

Future uncertainty

Very distant predictions may be less trustworthy.

Mathematical control

For continuing tasks, $\gamma < 1$ keeps bounded discounted returns finite.

Key idea

γ is not “the probability the future exists.” It is a modeling parameter controlling the value of delayed reward.

A Policy Is Not a Path

$$\pi(s) = a$$

Path

Intersection $\rightarrow$ Plaza $\rightarrow$ Library $\rightarrow$ Goal.
One assumed state sequence.

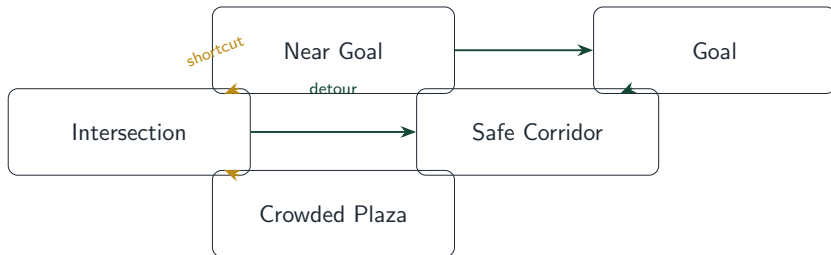
Policy

If crowd low $\rightarrow$ shortcut.
If crowd high $\rightarrow$ detour.
If localization poor $\rightarrow$ scan.

Key idea

A policy specifies what to do from every state the robot might actually reach.

A Tiny Campus MDP



Key idea

An MDP graph represents possible state transitions; probabilities and rewards determine how attractive each branch is.

Immediate Reward Can Choose the Wrong Action

Shortcut:

$$R = +6$$

Detour:

$$R = +2$$

A one-step agent prefers the shortcut immediately.

But if the shortcut often enters a terrible future state, the correct long-run decision can reverse.

Key idea

One-step expected utility becomes insufficient once the next state affects future opportunity.

Action Value Adds the Future

$$Q(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V(s')]$$

- ▶ T : how likely is each next state?
- ▶ R : what do we receive now?
- ▶ $V(s')$: how good is the future state we create?

Key idea

An action is good when it gives good reward now and tends to place the robot in valuable future states.

Bellman Optimality: The Core Recursion

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

Read it procedurally:

1. consider one candidate action;
2. consider every next state it can produce;
3. add immediate reward to discounted future value;
4. average by transition probability;
5. choose the action with the largest result.

Key idea

The value of a state is the value of making the best first move and then behaving optimally afterward.

Worked Comparison: Shortcut vs. Detour

Assume $\gamma = 0.9$.

Shortcut:

$$Q = 6 + 0.9[0.70(20) + 0.30(-25)] = 11.85$$

Detour:

$$Q = 2 + 0.9[0.95(17) + 0.05(-25)] = 15.41$$

$$15.41 > 11.85$$

Key idea

The detour loses on immediate reward but wins after future consequences are included.

What Is a Value Function?

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$$

$V^\pi(s)$ asks:

If I start in state s and follow policy π from now on, how much discounted reward should I expect?

$V^*(s)$ asks the same question assuming optimal behavior from now on.

Key idea

Values compress many possible future trajectories into one reusable number.

Value Iteration: Repeated Bellman Backups

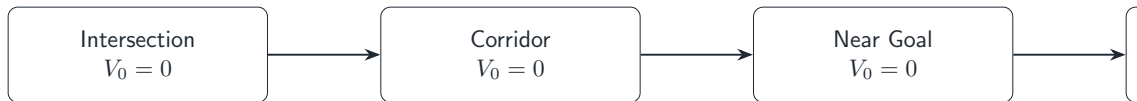
$$V_{k+1}(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V_k(s')]$$

1. initialize values, often $V_0(s) = 0$;
2. perform a Bellman backup for each state;
3. repeat until values stabilize sufficiently.

Key idea

Value iteration converts a long-horizon planning problem into repeated local updates.

Intuition: Reward Propagates Backward



Sweep 1 gives value to states near the goal. Later sweeps propagate that value farther backward.

Key idea

Dynamic programming reuses downstream value estimates instead of enumerating every complete route.

Extract the Policy from the Values

$$\pi^*(s) = \arg \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')]$$

Example policy:

- ▶ Intersection → Detour
- ▶ SafeCorridor → Forward
- ▶ CrowdedPlaza → Wait / Exit
- ▶ NearGoal → Deliver

Key idea

The value function scores states; the policy chooses the action that best exploits those values.

Policy Iteration: Another Route to the Same Goal

Policy evaluation

Hold a policy fixed and estimate how valuable every state is if the robot keeps following it.
Repeat until the policy no longer changes.

Policy improvement

At each state, check whether a different action would produce greater expected long-term value.

Key idea

Value iteration updates values directly; policy iteration alternates evaluating a policy and improving it.

What an MDP Quietly Assumes

Standard MDP planning assumes:

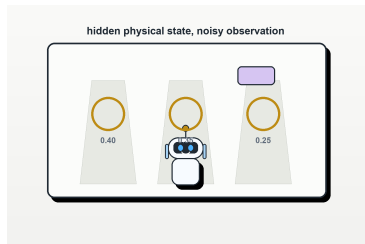
- ▶ the current modeled state is known;
- ▶ a transition model is available;
- ▶ the state/action space is manageable enough to represent values or policies.

Key idea

MDPs solve sequential uncertainty in outcomes—but standard MDPs still assume the current state is observable.

And Here Is the Problem: The Robot Does Not Know the True State

- ▶ the camera can miss pedestrians;
- ▶ LiDAR can be partially occluded;
- ▶ localization can drift;
- ▶ two hallways may look nearly identical;
- ▶ a door may be hidden around a corner.



Key idea

The state that matters for planning can be hidden even when the robot has excellent sensors.

Full Observability Was Doing More Work Than It Looked Like

An MDP policy is

$$\pi(s).$$

But the robot may only know

$$P(\textit{Corridor}A) = 0.55, \quad P(\textit{Corridor}B) = 0.45.$$

Which state should we plug into the MDP policy?

Key idea

If the current state is uncertain, a policy over physical states cannot be applied directly.

But Lecture 21 Already Gave Us the Missing Object

$$b_t(s) = P(S_t = s \mid o_{1:t}, a_{1:t-1})$$

The **belief state** is a probability distribution over possible physical states.

Example:

$$b_t = [0.55, 0.35, 0.10]$$

for Corridor A, Corridor B, and Plaza.

Key idea

State estimation and sequential decision making meet at the belief state.

Partially Observable Markov Decision Process

$$POMDP = (S, A, T, R, \Omega, O, \gamma)$$

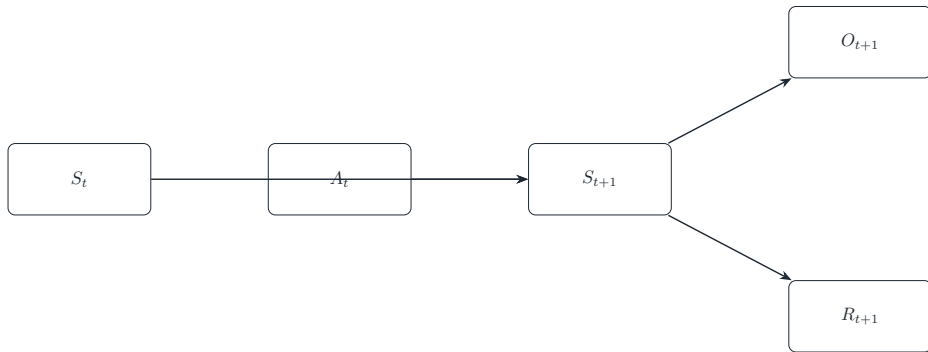
Relative to an MDP we add:

- ▶ Ω : the possible observations;
- ▶ $O(o \mid s', a)$: the observation model.

Key idea

A POMDP is an MDP where the agent acts without directly observing the underlying state.

The POMDP as a Dynamic Bayes Network



Key idea

The hidden state evolves, actions influence that evolution, and observations provide noisy evidence about the new state.

The Belief State Becomes the Planning State

Physical state:

where the robot actually is, whether the route is truly blocked, whether a door is truly open.

Belief state:

$$b = [0.55, 0.35, 0.10].$$

The controller now uses

$$\pi(b)$$

instead of $\pi(s)$.

Key idea

The robot plans from a probability distribution rather than pretending one hidden state is certainly true.

Belief Update: Predict, Then Correct

Prediction after action a_t :

$$b_{t+1}^-(s') = \sum_s T(s, a_t, s') b_t(s).$$

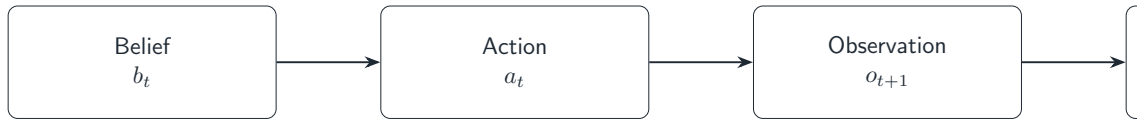
Correction after observation o_{t+1} :

$$b_{t+1}(s') \propto O(o_{t+1} \mid s', a_t) b_{t+1}^-(s').$$

Key idea

The POMDP belief update is the same predict–correct structure from filtering, now embedded inside decision making.

The Elegant Trick: A POMDP Becomes an MDP over Beliefs



The full action/observation history can be compressed into the current belief.

Key idea

The belief is a sufficient statistic for POMDP control—but that elegance creates a new computational problem.

Problem #1: The Belief Space Is Continuous

Suppose there are only two possible hidden states: A and B .

A belief is

$$b = [P(A), 1 - P(A)].$$

Every value

$$0 \leq P(A) \leq 1$$

defines a different belief state.

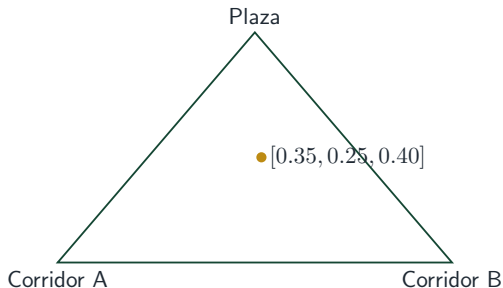
Key idea

Two physical states already create infinitely many possible beliefs.

With Three Hidden States, Beliefs Fill a Triangle

For three hidden states,

$$b = [p_1, p_2, p_3], \quad p_1 + p_2 + p_3 = 1.$$



Key idea

For n hidden states, beliefs live in a continuous $(n - 1)$ -dimensional probability simplex.

A POMDP Policy Maps Beliefs to Actions

Different belief regions can prefer different actions:

- ▶ highly confident in Corridor A → turn left;
- ▶ location ambiguous but uncertainty reducible → scan;
- ▶ likely near the plaza with high congestion → detour.

Key idea

The best action can change because the belief changes—even when the most likely physical state stays the same.

Problem #2: Future Observations Create Branching Contingencies

POMDP planning must reason about plans like:

Choose action a_1 . If observation o_1 arrives, do a_2 . If observation o_2 arrives instead, do a_3 .

So the future branches on:

- ▶ actions;
- ▶ stochastic physical transitions;
- ▶ stochastic observations;
- ▶ resulting belief updates.

Key idea

POMDP planning is contingency planning over both uncertain world evolution and uncertain future information.

How Fast Does That Branching Grow?

A rough action–observation branching count is

$$(|A||\Omega|)^H.$$

With four actions, three coarse observation categories, and horizon $H = 6$:

$$(4 \cdot 3)^6 = 12^6 \approx 2.99 \times 10^6.$$

And every branch carries a continuously valued belief distribution.

Key idea

Horizon, action branching, observation branching, and continuous beliefs compound each other.

So We End Up Back at a Familiar Question

Do we really need to enumerate every possible future?

Exact approach

Represent and evaluate every relevant branch of the future.

Monte Carlo approach

Sample plausible futures, observe their returns, and estimate which actions look best.

Key idea

The same tradeoff returns: give up exact enumeration to gain tractable approximate decision making.

Monte Carlo Rollouts for Decision Making

A hand-wavy online planner can:

1. sample a physical state from the current belief;
2. try a candidate action;
3. sample the next state and observation;
4. continue simulating a plausible future;
5. accumulate discounted reward;
6. repeat many times and average the return.

Key idea

Monte Carlo planning estimates long-term action value from sampled futures instead of summing over all futures exactly.

The Pieces We Already Learned Can Be Reused

Particle belief

Represent the current belief with particles instead of an exact table.

Monte Carlo futures

Sample transitions and observations to generate possible trajectories.

Tree search

Spend more simulation on promising choices rather than exploring everything uniformly.

Key idea

Approximate POMDP planning can combine sampled beliefs with Monte Carlo lookahead instead of constructing the entire belief-space solution.

The Unit Has Come Full Circle



Each new model relaxes an assumption from the previous one—then pays for that flexibility with more computation.

Key idea

AI repeatedly trades exactness, model simplicity, and computational cost to make rational decisions under real-world uncertainty.